

—— // AWKRS // ——

# awkrs – Language Reference

v0.4.20

---

## LANGUAGE REFERENCE

every builtin · function · variable · special file

2026-07-29



MENKE  
TECHNOLOGIES



MTPUBLISHING



MTOPENSOURCE

# Contents

|                                          |           |
|------------------------------------------|-----------|
| <b>String Functions</b>                  | <b>2</b>  |
| <b>Arithmetic Functions</b>              | <b>4</b>  |
| <b>I/O &amp; General Functions</b>       | <b>6</b>  |
| <b>Time Functions (gawk)</b>             | <b>7</b>  |
| <b>Bitwise Functions (gawk)</b>          | <b>8</b>  |
| <b>Array &amp; Type Functions (gawk)</b> | <b>9</b>  |
| <b>Special Variables</b>                 | <b>10</b> |
| <b>Keywords &amp; Control Flow</b>       | <b>13</b> |

# String Functions

11 topics

## length

```
length([s])
```

Length of string `s` (or `$0`), or element count of an array.

## substr

```
substr(s, m [, n])
```

`n`-char substring of `s` starting at position `m` (1-based).

## index

```
index(s, t)
```

1-based position of `t` in `s`, or 0 if not found.

## split

```
split(s, arr [, fs [, seps]])
```

Split `s` into `arr` on `fs`; returns the field count.

## sub

```
sub(re, repl [, target])
```

Replace first match of `re` in `target` (`$0`); returns 1/0.

## gsub

```
gsub(re, repl [, target])
```

Replace all matches of `re` in `target` (`$0`); returns count.

## match

```
match(s, re [, arr])
```

Set RSTART/RLENGTH to the match of `re` in `s`; returns position or 0.

## sprintf

```
sprintf(fmt, ...)
```

Format the arguments per `fmt` and return the string.

## tolower

```
tolower(s)
```

Copy of **s** with uppercase letters lowercased.

## toupper

```
toupper(s)
```

Copy of **s** with lowercase letters uppercased.

## gensub

```
gensub(re, repl, how [, target])
```

Non-destructive global/Nth substitution returning the new string.

# Arithmetic Functions

9 topics

## `sin`

```
sin(x)
```

Sine of  $x$  (radians).

## `cos`

```
cos(x)
```

Cosine of  $x$  (radians).

## `atan2`

```
atan2(y, x)
```

Arctangent of  $y/x$  in radians.

## `exp`

```
exp(x)
```

e raised to the power  $x$ .

## `log`

```
log(x)
```

Natural logarithm of  $x$ .

## `sqrt`

```
sqrt(x)
```

Square root of  $x$ .

## `int`

```
int(x)
```

Integer part of  $x$ , truncated toward zero.

## `rand`

```
rand()
```

Pseudo-random number in  $[0, 1)$ .

## srand

```
srand([x])
```

Seed the RNG with `x` (or time); returns the previous seed.

# I/O & General Functions

6 topics

## print

`print` — keyword

Write its arguments to output, separated by `DFS` and terminated by `ORS`.

## printf

```
printf(fmt, ...)
```

Format and print the arguments per `fmt`.

## getline

`getline` — keyword

Read the next record into `$0` or a variable from input, a file, or a command.

## close

```
close(file [, how])
```

Close an open file/pipe; returns its status.

## system

```
system(cmd)
```

Run `cmd` via the shell; returns its exit status.

## fflush

```
fflush([file])
```

Flush buffers for `file`, or all outputs if omitted.

# Time Functions (gawk)

3 topics

## sys`time`

```
systime()
```

Current time as seconds since the epoch (gawk).

## str`ftime`

```
strftime([fmt [, ts [, utc]])
```

Format timestamp `ts` per `fmt` (gawk).

## mk`time`

```
mktime(spec [, utc])
```

Convert a "YYYY MM DD HH MM SS [DST]" spec to a timestamp (gawk).

# Bitwise Functions (gawk)

6 topics

## and

```
and(v1, v2, ...)
```

Bitwise AND of the arguments (gawk).

## or

```
or(v1, v2, ...)
```

Bitwise OR of the arguments (gawk).

## xor

```
xor(v1, v2, ...)
```

Bitwise XOR of the arguments (gawk).

## compl

```
compl(v)
```

Bitwise complement of `v` (gawk).

## lshift

```
lshift(v, n)
```

`v` left-shifted by `n` bits (gawk).

## rshift

```
rshift(v, n)
```

`v` right-shifted by `n` bits (gawk).

# Array & Type Functions (gawk)

5 topics

## typeof

```
typeof(x)
```

Type of **x**: "scalar", "array", "untyped", ... (gawk).

## isarray

```
isarray(x)
```

1 if **x** is an array, else 0 (gawk).

## atsplit

```
atsplit(s, arr [, fpat [, seps]])
```

Split **s** into **arr** by the pattern **fpat** (gawk).

## asort

```
asort(src [, dst [, how]])
```

Sort array values; returns the element count (gawk).

## asorti

```
asorti(src [, dst [, how]])
```

Sort array indices; returns the element count (gawk).

# Special Variables

25 topics

## NR

**NR** — special variable

Total number of input records read so far.

## NF

**NF** — special variable

Number of fields in the current record.

## FS

**FS** — special variable

Input field separator (default " ").

## OFS

**OFS** — special variable

Output field separator (default " ").

## ORS

**ORS** — special variable

Output record separator (default "\n").

## RS

**RS** — special variable

Input record separator (default "\n").

## FILENAME

**FILENAME** — special variable

Name of the current input file.

## FNR

**FNR** — special variable

Record number within the current input file.

## RSTART

**RSTART** — special variable

Start position of the last match() (1-based), or 0.

## RLENGTH

**RLENGTH** — special variable

Length of the last match(), or -1.

## SUBSEP

**SUBSEP** — special variable

Subscript separator for multi-dimensional array keys.

## ARGC

**ARGC** — special variable

Count of command-line arguments in ARGV.

## ARGV

**ARGV** — special variable

Array of command-line arguments.

## ENVIRON

**ENVIRON** — special variable

Array of the process environment variables.

## CONVFMT

**CONVFMT** — special variable

Conversion format for numbers used as strings (default "%.6g").

## OFMT

**OFMT** — special variable

Output format for numbers in print (default "%.6g").

## IGNORECASE

**IGNORECASE** — special variable

When non-zero, regex and string comparisons ignore case (gawk).

## FIELDWIDTHS

**FIELDWIDTHS** — special variable

Space-separated fixed field widths for parsing (gawk).

## FPAT

**FPAT** — special variable

Regex describing field contents, as an alternative to FS (gawk).

## PROCINFO

**PROCINFO** — special variable

Array of process/runtime information (gawk).

## RT

**RT** — special variable

Text matched by RS for the current record (gawk).

## ERRNO

**ERRNO** — special variable

Description of the last getline/close/system error (gawk).

## TEXTDOMAIN

**TEXTDOMAIN** — special variable

Text domain for string translation (gawk).

## BINMODE

**BINMODE** — special variable

Binary I/O mode control (gawk).

## LINT

**LINT** — special variable

Dynamic control of lint warnings (gawk).

# Keywords & Control Flow

21 topics

## BEGIN

**BEGIN** — keyword

Special pattern whose action runs once before any input is read.

## END

**END** — keyword

Special pattern whose action runs once after all input is consumed.

## BEGINFILE

**BEGINFILE** — keyword

Pattern whose action runs before each input file is read (gawk).

## ENDFILE

**ENDFILE** — keyword

Pattern whose action runs after each input file is processed (gawk).

## function

**function** — keyword

Define a user function: **function** *name(params)* { *body* }.

## if

**if** — keyword

Conditional statement: **if** (*cond*) *stmt* with an optional **else** branch.

## else

**else** — keyword

The alternative branch taken when an **if** condition is false.

## while

**while** — keyword

Loop that repeats *stmt* while *cond* is true: **while** (*cond*) *stmt*.

## for

**for** — keyword

Loop: **for** (*init; cond; incr*) *stmt*, or **for** (*key in array*) *stmt*.

## do

**do** — keyword

Do-while loop: **do** *stmt* **while** (*cond*); the body runs at least once.

## break

**break** — keyword

Exit the innermost **for**, **while**, or **do** loop immediately.

## continue

**continue** — keyword

Skip to the next iteration of the innermost loop.

## next

**next** — keyword

Stop processing the current record and read the next one.

## nextfile

**nextfile** — keyword

Stop processing the current input file and advance to the next.

## exit

**exit** — keyword

Stop reading input and run **END**; **exit** [*expr*] sets the exit status.

## return

**return** — keyword

Return from a user function, optionally with a value: **return** [*expr*].

## delete

**delete** — keyword

Remove an array element (**delete** *arr*[*k*]) or clear it (**delete** *arr*).

## switch

**switch** — keyword

Multi-way branch on a value: **switch** (*expr*) { **case** ...: ... } (gawk).

## case

**case** — keyword

A labeled branch inside a **switch** statement (gawk).

## `default`

`default` — keyword

The fallback branch taken when no `case` matches in a `switch` (gawk).

## `in`

`in` — keyword

Array membership test (`key in arr`) or iteration (`for (key in arr)`).